МИНОБРНАУКИ РОССИИ

Санкт-Петербургский государственный

электротехнический университет

«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Построение и анализ алгоритмов»

Тема: **Расстояние Левенштейна**

Вар. 13

Студент гр. 4343 **Лапшин А.Е.**

Преподаватель **Жангиров Т.Р.**

Санкт-Петербург

2026 г.

Задание

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. *replace(ε, a, b)* – заменить символ a на символ b.
2. *insert(ε, a)* – вставить в строку символ a (на любую позицию).
3. *delete(ε, b)* – удалить из строки символ b.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A и B, а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B.

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A; третья строка – B.

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Вар. 13. Вывести не одно, а все редакционные предписания с минимальной стоимостью. Для одного из предписаний продемонстрировать его применение для преобразования 1-ой строки во 2-ую.

Предполагается, что для весов операций действует правило треугольника: если две последовательные операции можно заменить одной, то это не ухудшает общую цену.

При выполнении операций запрещается применять операции к символам, которые уже были получены в результате выполнения операций (т.е. строка преобразуется всегда только слева направо) — актуально для вариантов 3–6, 16 (упрощающее условие).

Выполнение работы

В данной лабораторной работе используется алгоритм ***Вагнера-Фишера.***

Алгоритм Вагнера-Фишера используется для вычисления редакционного расстояния между двумя строками с учётом заданных весов операций (вставка, удаление, замена) с помощью таблицы, в которой рассматриваются все возможные префиксы первой и второй строки. Данная таблица постепенно заполняется, шаг за шагом вычисляя минимальную стоимость преобразования для всё более длинных частей строк. Когда таблица полностью заполнена, значение в правой нижней ячейке и есть искомая минимальная стоимость преобразования первой строки во вторую целиком.

Используемые структуры

- matrix - таблица размером (len_S+1) × (len_T+1).matrix[i][j] хранит минимальную стоимость преобразования первых i символов строки S в первые j символов строки T.
- all_editorial_instructions - вектор для хранения всех найденных редакционных предписаний с минимальной стоимостью. Каждый элемент - структура editorial_instruction, содержащая строку операций и выровненную версию исходной строки.

Описание алгоритма

Инициализация. Задаётся таблица matrix размеро (len_S+1) × (len_T+1), где len_S - длина исходной строки S, а len_T - длина целевой строки T. Изначально таблица заполняется нулями. Первый столбец заполняется стоимостью последовательного удаления i символов: matrix[i][0] = i * cost_delete. Первая строка заполняется стоимостью последовательной вставки j символов: matrix[0][j] = j * cost_insert.

Основной цикл. Таблица заполняется построчно: для каждой ячейки matrix[i][j], если символы S[i-1] и T[j-1] совпадают, то данной ячейке присваивается значение ячейки по диагонали matrix[i-1][j-1], что соответствует отсутствию операции (стоимость 0). Если символы не совпали, то рассматриваются три операции:

- Удаление: matrix[i-1][j] + cost_delete
- Вставка: matrix[i][j-1] + cost_insert
- Замена: matrix[i-1][j-1] + cost_replace

Из этих трёх значений выбирается минимальное, которое и записывается в matrix[i][j].

Получение результата. После заполнения всей таблицы искомое редакционное расстояние находится в ячейке matrix[len_S][len_T]. Далее начинается восстановление последовательности операций: вызывается рекурсивная функция collectAllPathsc начальными координатами (len_S, len_T) и пустыми строками для накопления операций и символов. Из каждой позиции (i, j) рассматриваются возможные переходы назад:

- Совпадение / Замена - (i-1, j-1)
- Удаление - (i-1, j)
- Вставка - (i, j-1)

Для каждого перехода вычисляется ожидаемая стоимость. Если ожидаемая стоимость совпадает с фактическим значением matrix[i][j], то данный переход является частью оптимального пути. Функция рекурсивно вызывается для этой позиции с добавлением соответствующей операции (M, R, D, I) и символа (или пробела) в накапливаемые строки. При достижении ячейки (0, 0) накопленные строки переворачиваются и сохраняются в вектор all_editorial_instructions.

Вывод результата. После завершения сбора всех оптимальных предписаний программа выводит их количество и содержимое. Для демонстрации практического применения выбирается последнее найденное предписание, которое передаётся в функцию demonstrate_instruction. Данная функция выполняет пошаговое преобразование исходной строки S в целевую строку T в соответствии с последовательностью операций. На каждом шаге выводится:

- Номер шага
- Тип выполняемой операции
- Затрагиваемый символ и его позиция
- Текущее состояние строки после выполнения операции

Оценка временной сложности

В процессе обратного хода каждая ячейка таблицы matrix может быть посещена многократно в рамках различных оптимальных путей. В худшем случае, когда веса операций подобраны таким образом, что в каждой ячейке все три возможных перехода дают одинаковую минимальную стоимость, количество оптимальных предписаний растёт экспоненциально относительно размеров строк.

Тогда временная сложность в худшем случае: $\boldsymbol{O}(\ \mathbf{3}\text{^}(\boldsymbol{len_S} + \boldsymbol{len_T}) \cdot (\boldsymbol{len_S} + \boldsymbol{len_T}))$

Оценка пространственной сложности

Рекурсивная функция collectAllPaths использует стек вызовов, глубина которого не превышает len_S + len_T (максимальная длина одного пути). В каждый момент времени в стеке хранится информация об одном исследуемом пути: текущие координаты, накопленная строка операций и накопленная строка символов, каждая длиной до len_S + len_T.

Следовательно, объём памяти, занимаемый одним путём в стеке, составляет O(len_S + len_T).

Каждое предписание содержит две строки длиной до len_S + len_T. Учитывая, что количество предписаний в худшем случае достигает $\mathbf{3}\hat{}(\boldsymbol{len_S} + \boldsymbol{len_T})$, **итоговая пространственная сложность в худшем случае:** $\boldsymbol{O}(\mathbf{3}\hat{}(\boldsymbol{len_S} + \boldsymbol{len_T}) \cdot (\boldsymbol{len_S} + \boldsymbol{len_T}))$

Приложение А

Код программы Wagner-Fischer.cpp:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

int cost_replace, cost_insert, cost_delete;
string S, T;
int len_S, len_T;
vector<vector<int>> matrix;

struct editorial_instruction {
    string operations;
    string result_S;
};

vector<editorial_instruction> all_editorial_instructions;

void collectAllPaths(int i, int j, string operations, string
res_S) {

    if (i == 0 && j == 0) {
        reverse(operations.begin(), operations.end());
        reverse(res_S.begin(), res_S.end());
        all_editorial_instructions.push_back({operations,
res_S});

        cout << "Дошли до левого верхнего угла матрицы (0, 0)" <<
endl;
        cout << "Добавляем в all_editorial_instructions новый
случай:" << endl;
        cout << "   operations = " << operations << endl;
        cout << "   result_S = " << res_S << endl;
```

```
            return;
        }

        cout << "Проверяем три возможных перехода:" << endl;
        if (i > 0 && j > 0) {
            cout << "   Диагональный случай (i-1, j-1): операция M
(совпадение) или R (замена):" << endl;
            int expected_cost;
            char operate;
            if (S[i - 1] == T[j - 1]) {
                expected_cost = matrix[i - 1][j - 1];
                operate = 'M';
                cout << "      Это перация M - символы  S[i - 1] = "
<< S[i - 1] << " и T[j - 1] = " << T[j - 1] << " совпали" << endl;
                cout << "      Ожидаемая стоимость expected_cost = "
<< expected_cost << endl;
                cout << "      Нужная стоимость matrix[i][j] = " <<
matrix[i][j] << endl;
            } else {
                expected_cost = matrix[i - 1][j - 1] + cost_replace;
                operate = 'R';
                cout << "      Это перация R - символы  S[i - 1] = "
<< S[i - 1] << " и T[j - 1] = " << T[j - 1] << " не совпали" << endl;
                cout << "      Ожидаемая стоимость expected_cost = "
<< expected_cost << endl;
                cout << "      Нужная стоимость matrix[i][j] = " <<
matrix[i][j] << endl;
            }

            if (matrix[i][j] == expected_cost) {
                cout << "      \nНужная стоимость совпала с ожидаемой
стоимостью " << endl;
                cout << "      Значит, этот переход является частью
оптимального пути" << endl;
                cout << "      Добавляем операцию " << operate << " к
operations " << endl;
                cout << "      Добавляем символ S[i-1] = " << S[i-1]
<< " к res_S" << endl;
```

```
                cout << "      Рекурсивно идём в клетку (i-1, j-1)"
<< endl;


                collectAllPaths(i - 1, j - 1, operations + operate,
res_S + S[i - 1]);
            }
        }


        if (i > 0) {
            cout << "   Верхний случай (i-1, j): операция D
(удаление):" << endl;
            int expected_cost = matrix[i - 1][j] + cost_delete;
            cout << "      Ожидаемая стоимость expected_cost = " <<
expected_cost << endl;
            cout << "      Нужная стоимость matrix[i][j] = " <<
matrix[i][j] << endl;
            if (matrix[i][j] == expected_cost) {
                cout << "      \nНужная стоимость совпала с ожидаемой
стоимостью " << endl;
                cout << "      Значит, этот переход является частью
оптимального пути" << endl;
                cout << "      Добавляем операцию D к operations " <<
endl;
                cout << "      Добавляем символ S[i-1] = " << S[i-1]
<< " к res_S" << endl;
                cout << "      Рекурсивно идём в клетку (i-1, j)" <<
endl;


                collectAllPaths(i - 1, j, operations + 'D', res_S +
S[i - 1]);
            }
        }


        if (j > 0) {
            cout << "   Левый случай (i-1, j): операция I (вставка):"
<< endl;
            int expected_cost = matrix[i][j - 1] + cost_insert;
            cout << "      Ожидаемая стоимость expected_cost = " <<
expected_cost << endl;
            cout << "      Нужная стоимость matrix[i][j] = " <<
matrix[i][j] << endl;
```

```
            if (matrix[i][j] == expected_cost) {
                cout << "      \nНужная стоимость совпала с ожидаемой
стоимостью " << endl;
                cout << "      Значит, этот переход является частью
оптимального пути" << endl;
                cout << "      Добавляем операцию I к operations " <<
endl;
                cout << "      Добавляем пробел к res_S (так как в S
на этом месте ничего не было)" << endl;
                cout << "      Рекурсивно идём в клетку (i, j-1)" <<
endl;


                collectAllPaths(i, j - 1, operations + 'I', res_S + '
');
            }
        }
    }

    void demonstrate_instruction(editorial_instruction& instruct) {
        cout << "Операции: " << instruct.operations << "\n\n";

        string current = S;
        int pos = 0;
        int step = 0;

        cout << "Исходная строка: \"" << current << "\"\n\n";

        for (char op : instruct.operations) {
            cout << "Шаг " << (step + 1) << ": ";
            switch (op) {
                case 'M':
                    cout << "Совпадение '" << current[pos] << "'
(позиция " << pos << ")";
                    pos++;
                    break;
                case 'R':
                    cout << "Замена '" << current[pos] << "' на '" <<
T[step] << "'";
                    current[pos] = T[step];
```

```
                    pos++;
                    break;
                case 'D':
                    cout << "Удаление '" << current[pos] << "'
(позиция " << pos << ")";
                    current.erase(pos, 1);
                    break;
                case 'I':
                    cout << "Вставка '" << T[step] << "' на позицию "
<< pos;
                    current.insert(pos, 1, T[step]);
                    pos++;
                    break;
            }
            cout << "\n  Строка стала: \"" << current << "\"\n";
            step++;
        }

        cout << "\nПолученная строка: \"" << current << "\"\n";
        cout << "    Нужная строка: \"" << T << "\"\n";
    }

    int main() {
        cout << "Входные данные:" << endl;
        cout << "   первая строка - три числа: цена операции replace,
цена операции insert, цена операции delete" << endl;
        cout << "   вторая строка - S (какую преобразовываем)" <<
endl;
        cout << "   третья строка - T (в какую преобразовываем)" <<
endl;

        cin >> cost_replace >> cost_insert >> cost_delete;
        cin >> S >> T;
        len_S = S.length();
        len_T = T.length();

        matrix = vector<vector<int>>(len_S + 1, vector<int>(len_T +
1, 0));
```

```
        for (int i = 0; i <= len_S; i++) {
            matrix[i][0] = i * cost_delete;
        }
        for (int j = 0; j <= len_T; j++) {
            matrix[0][j] = j * cost_insert;
        }

        for (int i = 1; i <= len_S; i++) {
            for (int j = 1; j <= len_T; j++) {
                if (S[i - 1] == T[j - 1]) {
                    matrix[i][j] = matrix[i - 1][j - 1];
                } else {
                    int del_cost = matrix[i - 1][j] + cost_delete;
                    int ins_cost = matrix[i][j - 1] + cost_insert;
                    int rep_cost = matrix[i - 1][j - 1] +
cost_replace;
                    matrix[i][j] = min({del_cost, ins_cost,
rep_cost});
                }
            }
        }

        cout << "\nМинимальная стоимость предписания: " <<
matrix[len_S][len_T] << endl;



        collectAllPaths(len_S, len_T, "", "");


        cout << "\nМинимальная стоимость предписания: " <<
matrix[len_S][len_T] << endl;
        cout << "\nВсе найденные редакционные предписания с
минимальной ценой\n";
        cout << "Найдено предписаний: " <<
all_editorial_instructions.size() << "\n\n";


        for (size_t idx = 0; idx < all_editorial_instructions.size();
idx++) {
```

```
            cout << "Предписание №" << (idx + 1) << ":\n";
            cout << "  " << 
all_editorial_instructions[idx].operations << "\n";
            cout << "  " << all_editorial_instructions[idx].result_S 
<< "\n";
            cout << "  " << T << "\n\n";
        }



        cout << "\nПродемонстрируем последнее редакционное 
предписание:\n";
        if (!all_editorial_instructions.empty()) {
            demonstrate_instruction(all_editorial_instructions[all_ed
itorial_instructions.size() - 1]);
        }


        return 0;
    }
```

Приложение Б

Тестирование

Таблица 1 - Примеры тестовых случаев алгоритма поиска всех редакционных предписаний с минимальной стоимостью и с демонстрацией применения редакционного предписания.

№ п/п	Входные данные	Выходные данные	Комментарии
1.	1 1 1 abc abc	Минимальная стоимость предписания: 0 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 1 Предписание №1: MMM abc abc Продемонстрируем последнее редакционное предписание: Операции: MMM Исходная строка: "abc" Шаг 1: Совпадение 'a' (позиция 0) Строка стала: "abc" Шаг 2: Совпадение 'b' (позиция 1) Строка стала: "abc" Шаг 3: Совпадение 'c' (позиция 2) Строка стала: "abc" Полученная строка: "abc" Нужная строка: "abc"	Слова изначально совпадают
2.	1 1 1 abc xyz	Минимальная стоимость предписания: 3 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 1	Замена символов

		Предписание №1: RRR abc xyz Продемонстрируем последнее редакционное предписание: Операции: RRR Исходная строка: "abc" Шаг 1: Замена 'a' на 'x' Строка стала: "xbc" Шаг 2: Замена 'b' на 'y' Строка стала: "xyc" Шаг 3: Замена 'c' на 'z' Строка стала: "xyz" Полученная строка: "xyz" Нужная строка: "xyz"	
3.	1 1 1 a abcd	Минимальная стоимость предписания: 3 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 1 Предписание №1: MIII a abcd Продемонстрируем последнее редакционное предписание: Операции: MIII Исходная строка: "a" Шаг 1: Совпадение 'a' (позиция 0) Строка стала: "a" Шаг 2: Вставка 'b' на позицию 1 Строка стала: "ab"	Вставка символов

		Шаг 3: Вставка 'c' на позицию 2 Строка стала: "abc" Шаг 4: Вставка 'd' на позицию 3 Строка стала: "abcd" Полученная строка: "abcd" Нужная строка: "abcd"	
4.	1 1 1 dddabc abc	Минимальная стоимость предписания: 3 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 1 Предписание №1: DDDMMM dddabc abc Продемонстрируем последнее редакционное предписание: Операции: DDDMMM Исходная строка: "dddabc" Шаг 1: Удаление 'd' (позиция 0) Строка стала: "ddabc" Шаг 2: Удаление 'd' (позиция 0) Строка стала: "dabc" Шаг 3: Удаление 'd' (позиция 0) Строка стала: "abc" Шаг 4: Совпадение 'a' (позиция 0) Строка стала: "abc" Шаг 5: Совпадение 'b' (позиция 1) Строка стала: "abc" Шаг 6: Совпадение 'c' (позиция 2) Строка стала: "abc" Полученная строка: "abc"	Удаление символов

		Нужная строка: "abc"	
5.	1 1 1 entrance reenterable	Минимальная стоимость предписания: 5 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 2 Предписание №1: IIMMMIMMRRM ent rance reenterable Предписание №2: IMIMMIMMRRM e nt rance reenterable Продемонстрируем последнее редакционное предписание: Операции: IMIMMIMMRRM Исходная строка: "entrance" Шаг 1: Вставка 'r' на позицию 0 Строка стала: "rentrance" Шаг 2: Совпадение 'e' (позиция 1) Строка стала: "rentrance" Шаг 3: Вставка 'e' на позицию 2 Строка стала: "reentrance" Шаг 4: Совпадение 'n' (позиция 3) Строка стала: "reentrance" Шаг 5: Совпадение 't' (позиция 4) Строка стала: "reentrance" Шаг 6: Вставка 'e' на позицию 5 Строка стала: "reenterance" Шаг 7: Совпадение 'r' (позиция 6)	Прохождение задания со Stepik

		Строка стала: "reenterance" Шаг 8: Совпадение 'a' (позиция 7) Строка стала: "reenterance" Шаг 9: Замена 'n' на 'b' Строка стала: "reenterabce" Шаг 10: Замена 'c' на 'l' Строка стала: "reenterable" Шаг 11: Совпадение 'e' (позиция 10) Строка стала: "reenterable" Полученная строка: "reenterable" Нужная строка: "reenterable"	
6.	1 1 1 abacaba cababac	Минимальная стоимость предписания: 4 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 6 Предписание №1: IMMMRMDR abacaba cababac Предписание №2: IMMMRMRD abacaba cababac Предписание №3: DRMRMMMI abacaba cababac Предписание №4: RDMRMMMI abacaba cababac Предписание №5:	Строки с большим количеством редакционных предписаний

		IMMDDMMMI abacaba cababac Предписание №6: IMMMDDMMI abacaba cababac Продемонстрируем последнее редакционное предписание: Операции: IMMMDDMMI Исходная строка: "abacaba" Шаг 1: Вставка 'c' на позицию 0 Строка стала: "cabacaba" Шаг 2: Совпадение 'a' (позиция 1) Строка стала: "cabacaba" Шаг 3: Совпадение 'b' (позиция 2) Строка стала: "cabacaba" Шаг 4: Совпадение 'a' (позиция 3) Строка стала: "cabacaba" Шаг 5: Удаление 'c' (позиция 4) Строка стала: "cabaaba" Шаг 6: Удаление 'a' (позиция 4) Строка стала: "cababa" Шаг 7: Совпадение 'b' (позиция 4) Строка стала: "cababa" Шаг 8: Совпадение 'a' (позиция 5) Строка стала: "cababa" Шаг 9: Вставка " на позицию 6 Строка стала: "cababa" Полученная строка: "cababa" Нужная строка: "cababac"	

7.	3 1 2 abacaba cababac	Минимальная стоимость предписания: 6 Все найденные редакционные предписания с минимальной ценой Найдено предписаний: 2 Предписание №1: IMMDDMMMI abacaba cababac Предписание №2: IMMMDDMMI abacaba cababac Продемонстрируем последнее редакционное предписание: Операции: IMMMDDMMI Исходная строка: "abacaba" Шаг 1: Вставка 'c' на позицию 0 Строка стала: "cabacaba" Шаг 2: Совпадение 'a' (позиция 1) Строка стала: "cabacaba" Шаг 3: Совпадение 'b' (позиция 2) Строка стала: "cabacaba" Шаг 4: Совпадение 'a' (позиция 3) Строка стала: "cabacaba" Шаг 5: Удаление 'c' (позиция 4) Строка стала: "cabaaba" Шаг 6: Удаление 'a' (позиция 4) Строка стала: "cababa" Шаг 7: Совпадение 'b' (позиция 4) Строка стала: "cababa" Шаг 8: Совпадение 'a' (позиция 5) Строка стала: "cababa"	Результат для другой стоимости операций

		Шаг 9: Вставка " на позицию 6 Строка стала: "cababa" Полученная строка: "cababa" Нужная строка: "cababac"	